EXERCICE 04 / 4

ORCHESTRATION

CI/CD · GitHub Actions · Monitoring · WebSockets

Difficulte : ★★★★ Duree : 24 heures

Contexte pédagogique

EX04 assemble tout ce qui a ete construit en Section 4 dans un pipeline professionnel. GitHub Actions automatise les tests et le deploiement. Prometheus et Grafana surveillent les performances en temps reel. WebSocket connecte Redis Pub/Sub au Dashboard React pour des donnees IoT vraiment en direct. C'est la stack d'une startup tech en production.

Regle EX04 — Projet final Section 4

L'apprenant peut refactoriser l'ensemble du projet librement.
L'objectif est un pipeline CI/CD complet : chaque push sur main declenche les tests, construit l'image Docker et deploie automatiquement sur Railway ou VPS.

Competences ciblees

- GitHub Actions : workflow YAML, jobs, steps, secrets, variables d'environnement
- Tests automatises avec Jest + Supertest pour l'API
- Build et push d'image Docker vers GitHub Container Registry (GHCR)
- WebSocket avec ws ou socket.io : connexion bidirectionnelle en temps reel
- Integration Redis Pub/Sub → WebSocket → React Dashboard
- Prometheus : exposer des metriques depuis Node.js
- Variables de production : secrets GitHub, Railway environment variables

CONFIGURATION MOODLE — Parametres de l'activite

```
Type d'activite  : Devoir (Assignment) — PROJET FINAL SECTION 4
Titre            : EX04 — Orchestration : CI/CD + WebSocket + Monitoring
Prerequis        : EX03 valide >= 6/10
Delai            : 24 heures apres deblocage
Rendu            : URL Railway/VPS live + URL Dashboard React + Actions
GitHub vertes
Note max         : 10 points
Badge Moodle     : Badge 'Full Stack Engineer' — attribue a >= 8/10
Ressources       : Video GitHub Actions (20min) + Video WebSocket
(15min) + Guide CI/CD Railway
```

1 Pipeline CI/CD avec GitHub Actions

```
// YAML — .github/workflows/deploy.yml
```

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    name: Tests automatises
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_DB: cir_test
          POSTGRES_USER: testuser
          POSTGRES_PASSWORD: testpass
        ports: ['5432:5432']
        options: --health-cmd pg_isready --health-interval 10s

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm test
        env:
          DATABASE_URL :
postgres://testuser:testpass@localhost:5432/cir_test
          JWT_SECRET   : test_secret_for_ci

  deploy:
    name: Deploiement production
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v4
      - name: Deploy sur Railway
        uses: bervProject/railway-deploy@main
        with:
          railway_token: ${{ secrets.RAILWAY_TOKEN }}
          service: api-iot
```

- Le job deploy s'execute SEULEMENT si le job test reussit (needs: test)
- Les secrets GitHub (RAILWAY_TOKEN) sont configures dans Settings → Secrets → Actions
- if: github.ref == 'refs/heads/main' : deployer seulement sur la branche main, pas sur les PRs

2 Tests automatises avec Jest et Supertest

```
// JavaScript — Tests d'integration pour l'API
// npm install --save-dev jest supertest

// tests/sensors.test.js
const request = require('supertest');
const app     = require('../src/app');

let authToken;

// Se connecter avant tous les tests
beforeAll(async () => {
  // Creer un utilisateur de test et se connecter
  await request(app).post('/api/auth/register')
    .send({ name: 'Test', email: 'test@cir.ci', password: 'password123'
});

  const loginRes = await request(app).post('/api/auth/login')
    .send({ email: 'test@cir.ci', password: 'password123' });

  authToken = loginRes.body.token;
});

describe('GET /api/sensors', () => {
  it('retourne 401 sans token', async () => {
    const res = await request(app).get('/api/sensors');
    expect(res.status).toBe(401);
  });

  it('retourne 200 avec token valide', async () => {
    const res = await request(app).get('/api/sensors')
      .set('Authorization', `Bearer ${authToken}`);
    expect(res.status).toBe(200);
    expect(res.body.success).toBe(true);
    expect(Array.isArray(res.body.data)).toBe(true);
  });
});

describe('POST /api/sensors', () => {
  it('cree un capteur avec donnees valides', async () => {
    const res = await request(app).post('/api/sensors')
      .set('Authorization', `Bearer ${authToken}`)
      .send({ name: 'Capteur Test', type: 'temperature', value: 25.5 });
    expect(res.status).toBe(201);
    expect(res.body.data.name).toBe('Capteur Test');
  });

  it('retourne 400 avec type invalide', async () => {
    const res = await request(app).post('/api/sensors')
      .set('Authorization', `Bearer ${authToken}`)
      .send({ name: 'Test', type: 'invalid_type' });
    expect(res.status).toBe(400);
  });
});
```

- Supertest simule des requetes HTTP sans demarrer un vrai serveur — les tests sont rapides
- beforeAll : une seule connexion pour tous les tests — pas besoin de se reconnecter a chaque test
- Ecrire les tests AVANT le code (TDD) ou au minimum en meme temps — jamais apres

3 WebSocket — Donnees IoT en temps reel

Combiner Redis Pub/Sub (EX02) avec WebSocket pour pousser les nouvelles mesures vers le Dashboard React sans que le client ait a faire des requetes repetees (polling).

```
// JavaScript — Serveur WebSocket integre a Express
// npm install ws
const http      = require('http');
const WebSocket = require('ws');
const app       = require('./app');
const redis     = require('./config/redis');

const server = http.createServer(app);
const wss    = new WebSocket.Server({ server });

// Map des clients connectes par userId
const clients = new Map();

wss.on('connection', (ws, req) => {
  // Extraire et verifier le JWT depuis le query param
  const token  = new URL(req.url,
'ws://localhost').searchParams.get('token');
  const userId = verifyJWT(token)?.id;
  if (!userId) return ws.close(4001, 'Token invalide');

  // Enregistrer le client
  if (!clients.has(userId)) clients.set(userId, new Set());
  clients.get(userId).add(ws);

  ws.on('close', () => clients.get(userId)?.delete(ws));
});

// Abonnement Redis : recevoir les nouvelles mesures
const subscriber = redis.duplicate();
subscriber.psubscribe('sensor:*:readings');

subscriber.on('pmessage', (pattern, channel, message) => {
  const reading = JSON.parse(message);
  const userId  = reading.userId;

  // Envoyer aux clients WebSocket de cet utilisateur
  clients.get(userId)?.forEach(ws => {
    if (ws.readyState === WebSocket.OPEN) {
      ws.send(JSON.stringify({ type: 'NEW_READING', data: reading }));
    }
  });
});
```

```
// Dans React Dashboard :
// const ws = new WebSocket(`wss://api.railway.app?token=${token}`);
// ws.onmessage = (event) => {
//   const { type, data } = JSON.parse(event.data);
//   if (type === 'NEW_READING') setSensors(prev => [...prev, data]);
// };
```

- psubscribe avec un pattern 'sensor:*:readings' : s'abonner a tous les canaux qui matchent
- Map des clients par userId : chaque utilisateur ne recoit que ses propres donnees
- Dans React : remplacer le setInterval de polling par une connexion WebSocket permanente

4 Metriques Prometheus et finalisation

```
// JavaScript — Exposer des metriques pour Prometheus
// npm install prom-client
const promClient = require('prom-client');

// Metriques automatiques (CPU, memoire, event loop)
promClient.collectDefaultMetrics();

// Compteur personnalise : nombre de requetes API
const httpRequestsTotal = new promClient.Counter({
  name   : 'http_requests_total',
  help   : 'Total des requetes HTTP',
  labelNames: ['method', 'route', 'status'],
});

// Histogramme : temps de reponse
const httpDuration = new promClient.Histogram({
  name    : 'http_request_duration_seconds',
  help    : 'Duree des requetes HTTP',
  buckets : [0.001, 0.01, 0.05, 0.1, 0.5, 1, 2],
});

// Middleware pour mesurer chaque requete
app.use((req, res, next) => {
  const end = httpDuration.startTimer();
  res.on('finish', () => {
    httpRequestsTotal.inc({ method: req.method, route: req.route?.path ||
req.path, status: res.statusCode });
    end({ method: req.method, route: req.route?.path || req.path });
  });
  next();
});

// Route metriques pour Prometheus
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', promClient.register.contentType);
  res.end(await promClient.register.metrics());
});
```

- Prometheus scrappe la route /metrics toutes les 15 secondes pour collecter les metriques

- Grafana se connecte a Prometheus et visualise les metriques dans des dashboards
- Ajouter prometheus et grafana comme services dans docker-compose.yml